Physics-Informed Neural Networks for Mechanics: Reproduction, Diagnosis and Improvements

Daruru Bhuvan Sai (PES1UG24AM356) and **Vismay H S** (PES1UG24AM325)

UE24CS352A Machine Learning mini-project, PES University · 13 October 2026

At a glance

We reproduced all three experiments of the base paper over 5 to 10 random seeds, found the cause of each experiment's main error, and fixed it. The fixes cut the error 30 to 200 times or made a failing method reliable, and every fix trains faster than the paper's setup.

Problem	Paper's method, as we reproduced it	Our change	Result
Damped oscillator: find damping c and stiffness k from data	Finds both within 1% in 3 of 5 seeds; 699 s per run	Two-stage warm start: fit the data first, read c and k off the fit, then train the PINN	10 of 10 seeds; 23 s per run (30x faster)
Soil consolidation: pore pressure over depth and time	Relative L2 error 1.4e-2; predicted pressure overshoots to 1.11 (physical maximum is 1)	Corner-enriched output: an exact erf term carries the jump at t = 0	Relative L2 error 6.4e-5 (about 200x lower); no overshoot; 7x faster
Heat conduction in a square plate	Relative L2 error 1.3e-3; largest error on the edges; plate dips below 0 °C	Exact boundary condition (distance function), then exact corner terms	Relative L2 error 4.2e-5 (30x lower); max error 3.8e-3 down to 2.7e-5 (140x)
Oscillator with noisy, sparse data (not tested in the paper)	Not tested	Benchmark against a classical fit, a plain neural network and the Cramer-Rao bound	Classical fit is best in every setting; PINNs come close only with dense data

All errors are medians over seeds, measured against exact solutions that we derived and verified independently.

1. The problem

Physics-informed neural networks (PINNs) train a network to obey a differential equation instead of fitting labelled data alone. Our base paper, Zhang, Chen and Yang, *Data Driven Solutions and Discoveries in Mechanics Using Physics Informed Neural Network* (Stanford CS229), applies them to three mechanics problems. It reports one run of each and calls the match good. We asked three questions: does the result hold across seeds, where is the remaining error and why, and can it be removed?

Paper section	Problem	Type	Equation and conditions
4.1	One-dimensional (Terzaghi) consolidation of a soil layer	Forward: predict pressure p(x, t)	p_t = p_xx on 0 < x < 1, 0 < t ⩽ 0.5; p = 0 at the drained top, p_x = 0 at the sealed base, p = 1 at t = 0
4.2	Steady heat conduction with a uniform source	Forward: predict temperature T(x, y)	T_xx + T_yy + 1 = 0 on the square $(-1, 1)^2$; T = 0 on the edges
4.3	Damped mass-spring oscillator	Inverse: find c and k	u″ + c u′ + k u = 0, u(0) = u′(0) = 1, t in [0, 15]; 50 measurements of u; true c = 0.4, k = 4; start c = 0.2, k = 2

The paper also raises improvements it never tests: enforcing the boundary condition exactly, and adaptive placement of training points. We tested both and added a study of noisy and sparse data, which the paper does not cover.

2. Method

We built one PINN engine in PyTorch and used it for every experiment, with the paper's networks, point counts, loss weights and learning rates.

How a PINN is trained. A network N maps coordinates to the solution. Automatic differentiation gives its derivatives, so the equation's residual can be evaluated anywhere. Training minimises a weighted sum of mean-squared terms:

> **$L = w_PDE \cdot mean(r^2) + w_BC \cdot mean((N - g)^2) + w_IC \cdot mean((N - p_0)^2) + w_data \cdot mean((N - u_obs)^2)$**
>
> r is the equation's residual at the collocation points; g, p_0 and u_obs are the boundary values, initial values and measurements.

In the inverse problem, c and k are trainable parameters next to the network weights.

Engine design.

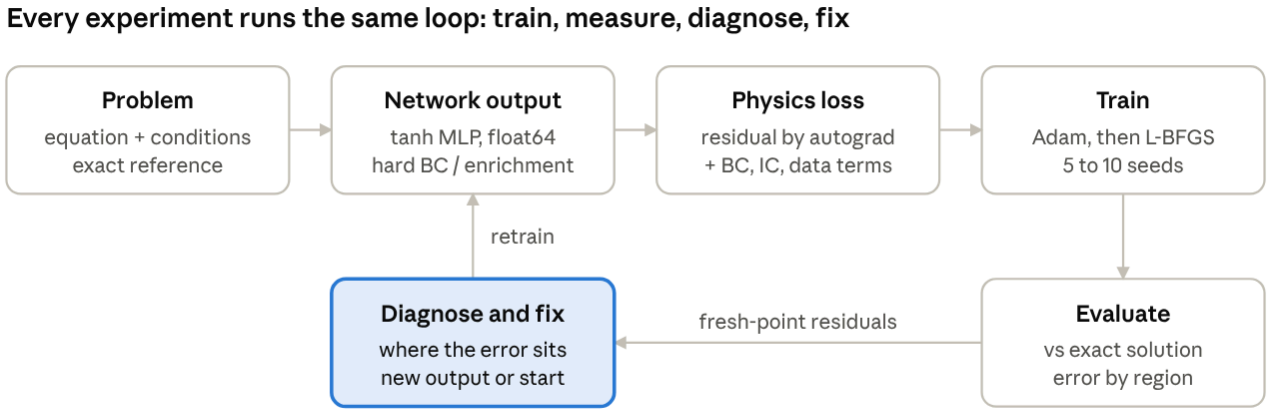
- Fully connected tanh network. tanh has useful second derivatives; ReLU does not.
- Inputs scaled to [-1, 1]; Xavier initialisation.
- Adam first to leave the random start, then L-BFGS to converge. If L-BFGS diverges, the last good weights are restored.
- float64 throughout. L-BFGS needs the precision near convergence, and on our RTX 4050 a step cost the same as float32 (12.2 vs 12.6 ms).
- Each loss term is logged separately, so a failure can be traced to the equation, boundary, initial condition or data.

Problem	Network (width x layers)	Training points	Optimiser (paper)	Learning rate
Oscillator	32 x 3	50 measurements, 100 collocation	Adam 100k	5e-4
Consolidation	50 x 5	500 collocation, 250 boundary, 125 initial	Adam 50k + L-BFGS	1e-3
Heat	50 x 4	1,500 collocation, 500 boundary	Adam 50k + L-BFGS	5e-4

How we evaluate.

- **Several seeds.** Every configuration runs over 5 seeds, and over 10 for the oscillator fixes. We report the median and the worst run, because a single run can mislead: one shortcut was 5/5 on five seeds but 8/10 on ten.
- **Exact references.** Oscillator: closed form. Consolidation: the method-of-images (erfc) solution, checked against the paper's Fourier series. Heat: the classical series solution, checked against our own finite-difference solver. The paper states that no analytical solution exists for the heat problem; one does.
- **Where the error sits.** We report relative L2 error and max error, plus the error near corners and edges and in early time. We also compare the equation residual on 20,000 fresh points against the training points; a large gap means the training loss is hiding an

error.



experiment pipeline · one engine for all three problems

Each improvement in this report came from one pass of this loop: the evaluation showed where the error sat, and the fix changed the network's output (consolidation, heat) or its starting point (oscillator) to remove it.

3. Damped oscillator: finding c and k

Reproduction. With the paper's settings, only 3 of 5 seeds find both parameters within 1%, and they need a median of 71,000 iterations. The paper's single run (c = 0.40125, k = 3.9933) was one of the good ones. k is always found; c is the problem.

Diagnosis. At every point in training, c and k equal the values implied by the curve the network currently draws. We verified this with a least-squares fit on the network's own derivatives.

- The network learns the signal from left to right. Until it reaches the end, the tail is flat, and a flat tail looks like heavy damping (c near 1).
- The late, small oscillations encode c but hold only 1% of the data's energy, so the data term barely pulls on them.
- u = 0 satisfies the equation exactly, so a flat tail costs the physics term nothing.
- c and k start wrong (0.2 and 2), so early on the physics term pulls the curve toward the wrong solution.

Fix: two-stage warm start.

1. Fit the network to the measurements and initial condition only. No physics yet, so nothing rewards a flat tail.
2. Read c and k off the fitted curve by least squares on its derivatives.
3. Train the full PINN with c and k frozen at those values.

4. Release c and k and train everything jointly.

Results (10 seeds; the paper row is our 5-seed reproduction):

Method	Seeds within 1%	c error, median (worst)	k error, median (worst)	Time per run
Paper (Adam 100k)	3/5	9.1e-3 (4.0e-2)	1.0e-3 (4.1e-3)	699 s
Adam 20k + L-BFGS	8/10	7.3e-3 (1.1e-1)	5.5e-4 (4.1e-3)	59 s
Two-stage, without step 2	7/10	7.3e-4 (5.2e-2)	4.6e-4 (1.0e-2)	22 s
Two-stage, without step 3	10/10	1.9e-3 (7.2e-3)	1.8e-4 (1.0e-3)	23 s
Two-stage (full)	**10/10**	**1.4e-3 (5.6e-3)**	**4.4e-4 (1.1e-3)**	**23 s**

The ablations show that reading c and k off the data fit (step 2) is the key ingredient: without it, success drops to 7/10. Freezing them first (step 3) is a small safety margin; without it we got 10/10 on one machine and 9/10 on another. Step 2 alone already gives c within 0.33% (median).

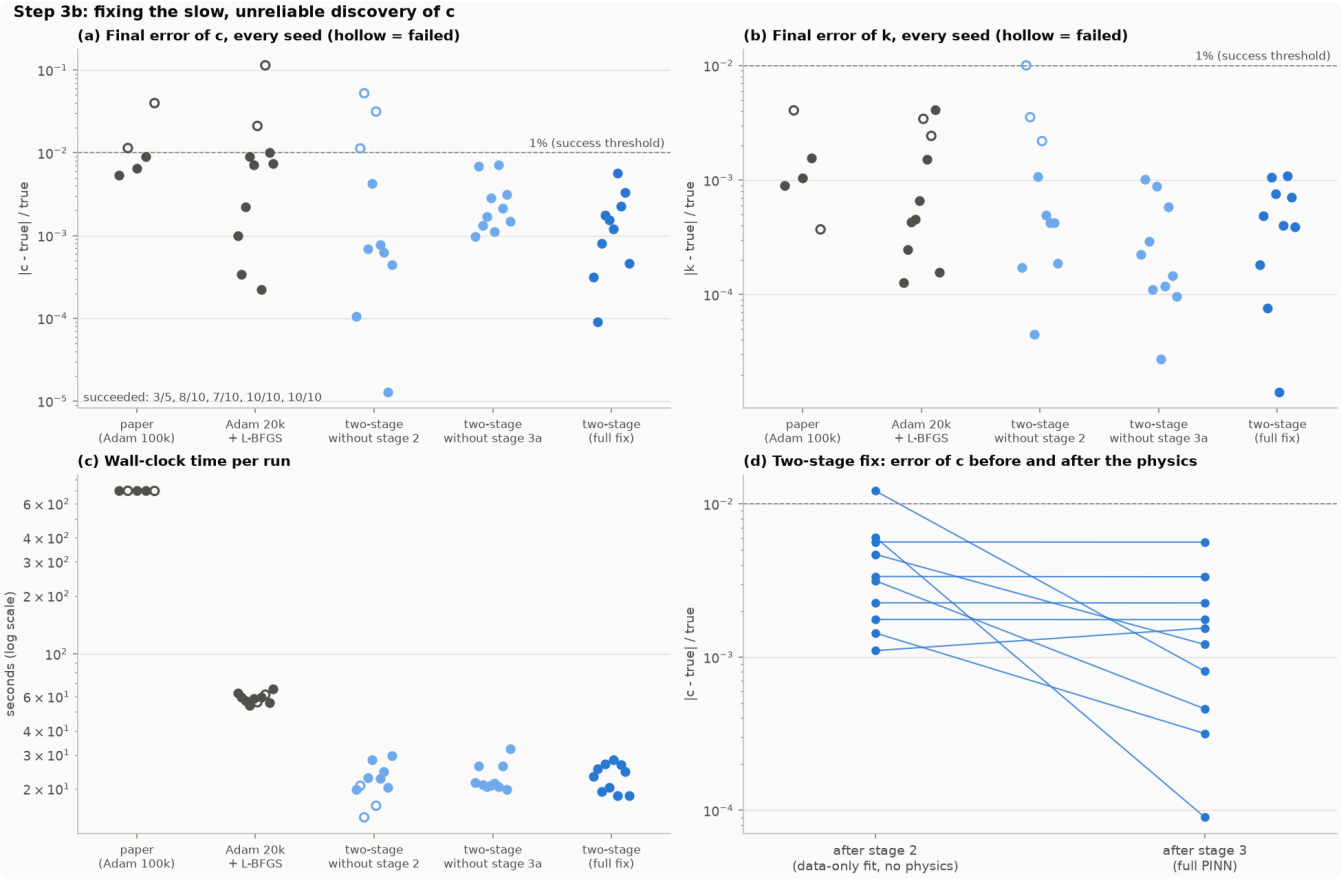


Figure 1. Oscillator. (a, b) Final error of c and k for every seed; hollow markers failed. (c) Time per run. (d) The two-stage method's c error after the data-only fit and after the full PINN.

4. Consolidation: the error at the corner

Reproduction. The paper reports a "perfect match" with a max error of about 0.045. Over 5 seeds the median max error (for $t \geq 0.01$) is 0.061, and the worst is 0.166. The predicted pressure overshoots to 1.11, above the physical maximum of 1.

Diagnosis. At $x = 0$, $t = 0$ the pressure must drop from 1 to 0 instantly, so the exact solution has an infinitely steep front at that corner.

- The first 2% of the time window ($t < 0.01$) holds 11 of the 500 random training points but 59% of the squared error.
- On fresh points, the equation residual is 49, against 5.5e-6 at the training points. The training loss looks converged while the solution is wrong near the corner.
- The paper's last 40k Adam iterations add little: its loss curve is flat after about 5k.

Fixes compared.

- **Adaptive sampling (RAD)** re-draws training points where the residual is largest. This is the "adaptive re-meshing" the paper lists as future work.
- **Corner-enriched output (ours).** Write the output as $p = \mathrm{erf}(x / 2\sqrt{t}) + x\,t\,N(x, t)$. The erf term is the exact solution for a layer with no base; it carries the jump. The network learns only the smooth effect of the base. This extends the paper's own trick of multiplying by x.

Method	Relative L2 error, median (worst)	Max error, $t \geq 0.01$	Highest pressure (physical max 1)	Residual on fresh points	Time per run
Paper (Adam 50k)	1.4e-2 (4.3e-2)	6.1e-2	1.108	4.9e+1	1,143 s
Baseline (Adam 10k)	1.7e-2 (3.2e-2)	9.7e-2	1.191	1.9e+2	433 s
Adaptive sampling (RAD)	1.3e-2 (3.8e-2)	3.3e-2	1.076	3.3e+1	395 s
Corner-enriched output	**6.4e-5 (9.6e-5)**	**2.9e-4**	**1.000**	**8.4e-7**	**168 s**
Enriched + RAD	5.4e-5 (8.0e-5)	2.1e-4	1.000	3.6e-7	160 s

RAD triples the points near the corner (11 to 37) but barely helps: no placement of points lets a smooth network represent an infinitely steep front. The enriched output removes that need. It cuts the error about 200-fold, removes the overshoot and trains 7 times faster than the paper's setup.

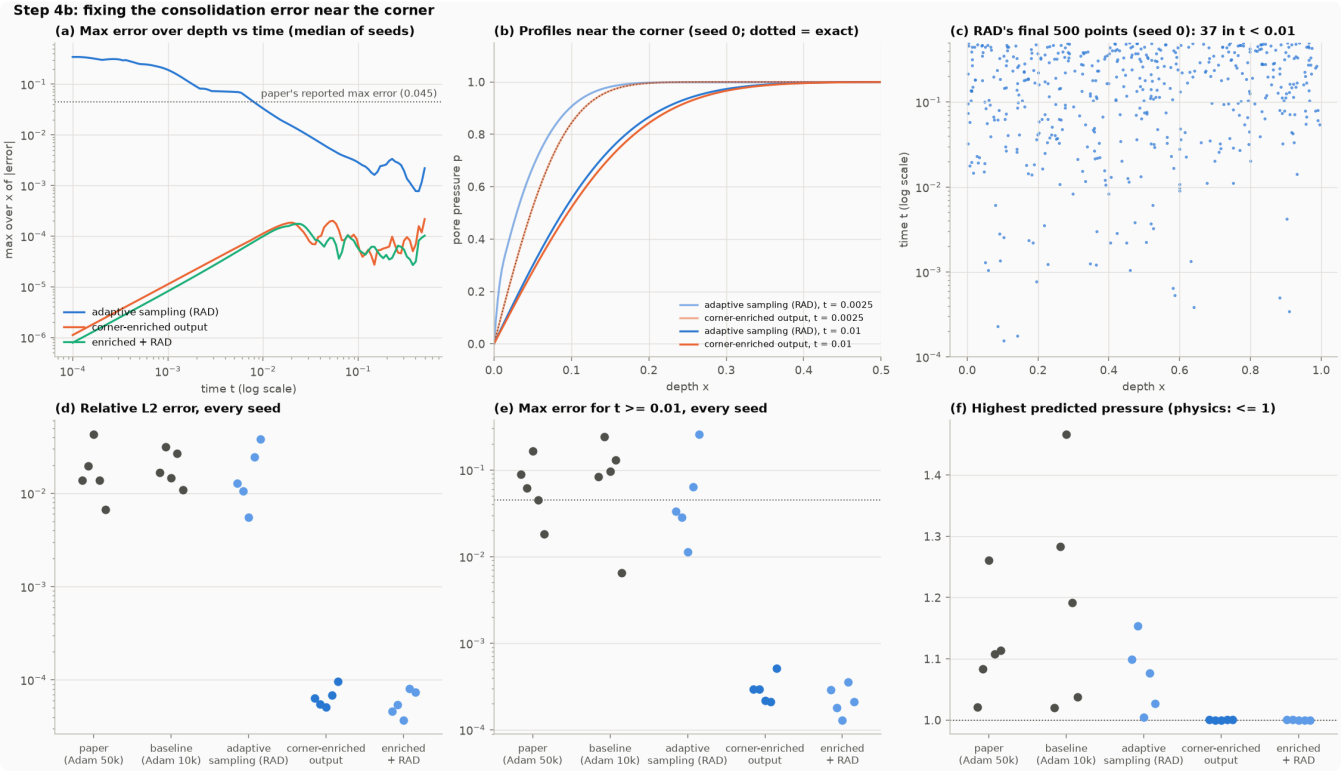


Figure 2. Consolidation. (a) Max error over depth against time: RAD stays above the paper's reported error until t ≈ 0.01; the enriched output stays at about 2e-4 or less throughout. (b) Profiles near the corner. (c) Where RAD put its points. (d–f) Error and highest pressure for every seed.

5. Heat conduction: edges, then corners

Reproduction. The paper's network enforces T = 0 on the edges only as a loss penalty (a "soft" boundary condition). It reports a max error of about 5.2e-3, on the edges. We measured 3.8e-3 (median), also on the edges. 60% of the squared error lies within 0.05 of an edge (10% of the area), and the plate dips to -3.8e-3, below its physical minimum of 0.

First fix: exact boundary condition. Write $T = (1 - x^2)(1 - y^2)\,N(x, y)$, so the boundary condition holds by construction. The paper suggests this but does not try it. It cuts the error about 30-fold, and the plate no longer goes below 0.

What still limits every PINN: the corners. With the hard boundary condition, the residual on fresh points is about 100 times the training loss, concentrated at the four corners. The cause is a contradiction in the problem itself:

- Along the top edge T = 0, so T_xx = 0. Along the side edge T_yy = 0, so the equation forces T_xx = -1.
- At the corner, T_xx would have to be both. The exact solution resolves this with a direction-dependent value, T_xx → -2θ/π, where θ is the approach angle.
- A smooth network has one value at each point, so it must break something. The soft-boundary network breaks T = 0: its largest error sits at a corner in 5 of 5 seeds. The hard-boundary network breaks the equation: its residual is exactly 1 at each corner, whatever its weights.

The direction-dependent T_xx and the residual of exactly 1 are properties of the problem, not of training. Our unit tests check both without training a network.

Second fix: corner terms. At each corner we add the exact local solution of a 90° corner, derived by hand from the corner alone, not from the series:

> $$u = r^2 \left[-1/4 + \cos(2\theta)/4 - \left(\ln r \cdot \sin(2\theta) + \theta \cdot \cos(2\theta) \right) / \pi \right]$$
>
> r and θ are polar coordinates from the corner. u is zero on both edges and gives T_xx = −2θ/π exactly.

Each term is faded out away from its corner and still vanishes on every edge. The network learns only a smooth remainder. This is the same idea as the erf term in consolidation.

Configuration	Relative L2 error, median (worst)	Max error	Max error's distance to the nearest corner	Squared error within 0.15 of a corner (1.8% of area)	Fresh / training residual	Time per run
Paper: soft BC, Adam 50k	1.3e-3 (1.6e-3)	3.8e-3	on the edge	not measured	2.2x	1,983 s
Soft BC, Adam 5k	1.5e-3 (2.5e-3)	4.8e-3	0.00 (at a corner)	26%	1.5x	356 s
Hard BC	4.2e-5 (7.3e-5)	7.7e-5	0.06	18%	97.7x	489 s
Hard BC + corner terms	**3.9e-5 (1.1e-4)**	**2.7e-5**	**0.87**	**2%**	**1.8x**	**945 s**

The corner terms remove the corner hotspot: the max error moves away from the corners and falls a further 3 times. The training loss can be trusted again (fresh / training residual 1.8x instead of 98x). The average error barely changes and training takes about twice as long. Rerunning the earlier configurations reproduced their errors bit for bit.

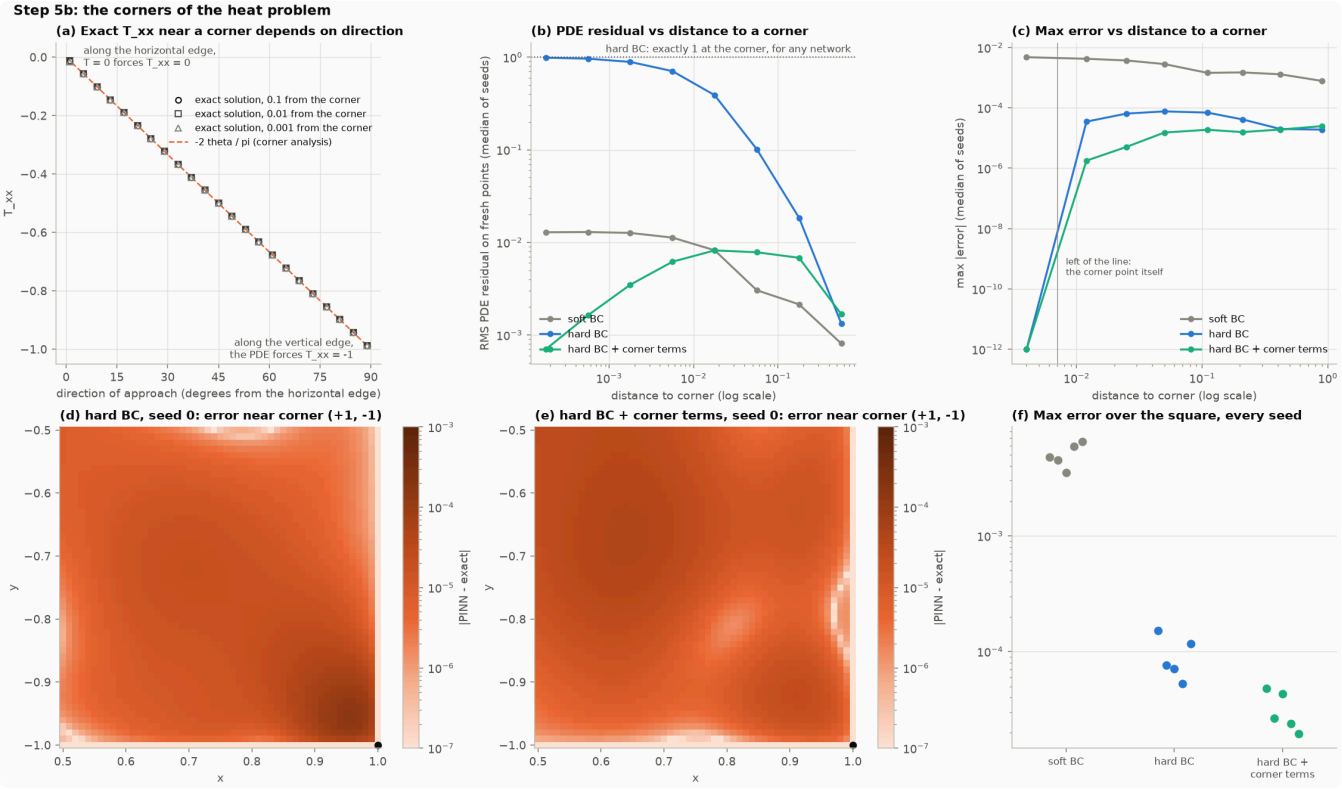


Figure 3. Heat corners. (a) The exact T_xx depends on the approach direction, matching -2θ/π. (b) The hard-BC residual rises to exactly 1 at the corner; the corner terms remove the rise. (c) Max error against distance to a corner. (d, e) Error near one corner without and with the corner terms. (f) Max error for every seed.

6. Noisy and sparse data

The paper tests parameter discovery only on clean data at 50 points and compares it with nothing. We tested how far it holds and against what.

Setup.

- **Data settings.** 50 points over [0, 15] (the paper's setting); 15 points over [0, 15] (fewer than 3 per oscillation); 50 points over [0, 5] only.
- **Noise.** Gaussian noise of 0%, 1%, 3% and 10% of the initial displacement, with 10 random draws per case.
- **Methods.** Every method sees the same data and starts from c = 0.2, k = 2:
  - Classical: fit c and k so the numerical ODE solution matches the data.
  - Plain neural network: fit the data, then read c and k off the fitted curve's derivatives.
  - Standard PINN: Adam 20k + L-BFGS.
  - Two-stage PINN: our method from section 3.

- **Benchmark.** The Cramer-Rao bound gives the smallest error any unbiased method can reach at each noise level. With 3% noise even an ideal method misses c by 1.3% half the time, so "within 1%" stops being a fair target.

Method	Clean data: runs within 1%	c error at 1% noise: 50 points	15 points	50 points, t ⩽ 5 only	Time per dataset
Ideal (Cramer-Rao bound)		4.3e-3	8.1e-3	4.4e-3	
Classical ODE fit	30/30	5.5e-3	1.2e-2	4.6e-3	0.2 s
Plain neural network	10/30	6.8e-3	4.8e-1	1.6e-2	13 s
Standard PINN	26/30	8.6e-3	1.5e-2	3.3e-2	61 s
Two-stage PINN	23/30	4.6e-3	2.7e-1	6.5e-3	38 s

c errors are relative medians over 10 draws.

Findings.

- **The classical fit wins everywhere.** It is fastest and stays close to the bound in every setting. When the equation is known and the parameters are few, a PINN is not the best tool.
- **The two-stage PINN is the best neural method on dense data.** At 50 points it sits within 7% of the bound at 1% noise.
- **It fails on sparse data.** With 15 points, the data-only fit in step 1 is poor, so the warm start points to the wrong c and k. The standard PINN is the better neural method there.
- **At 10% noise every neural method degrades.** Their c errors land 3 to 6 times above the bound, while the classical fit stays within 1.5 times.
- **Physics helps extrapolation.** With clean data only up to t = 5, the plain network's curve over [0, 15] is off by 83%; both PINNs stay within 13 to 22%.

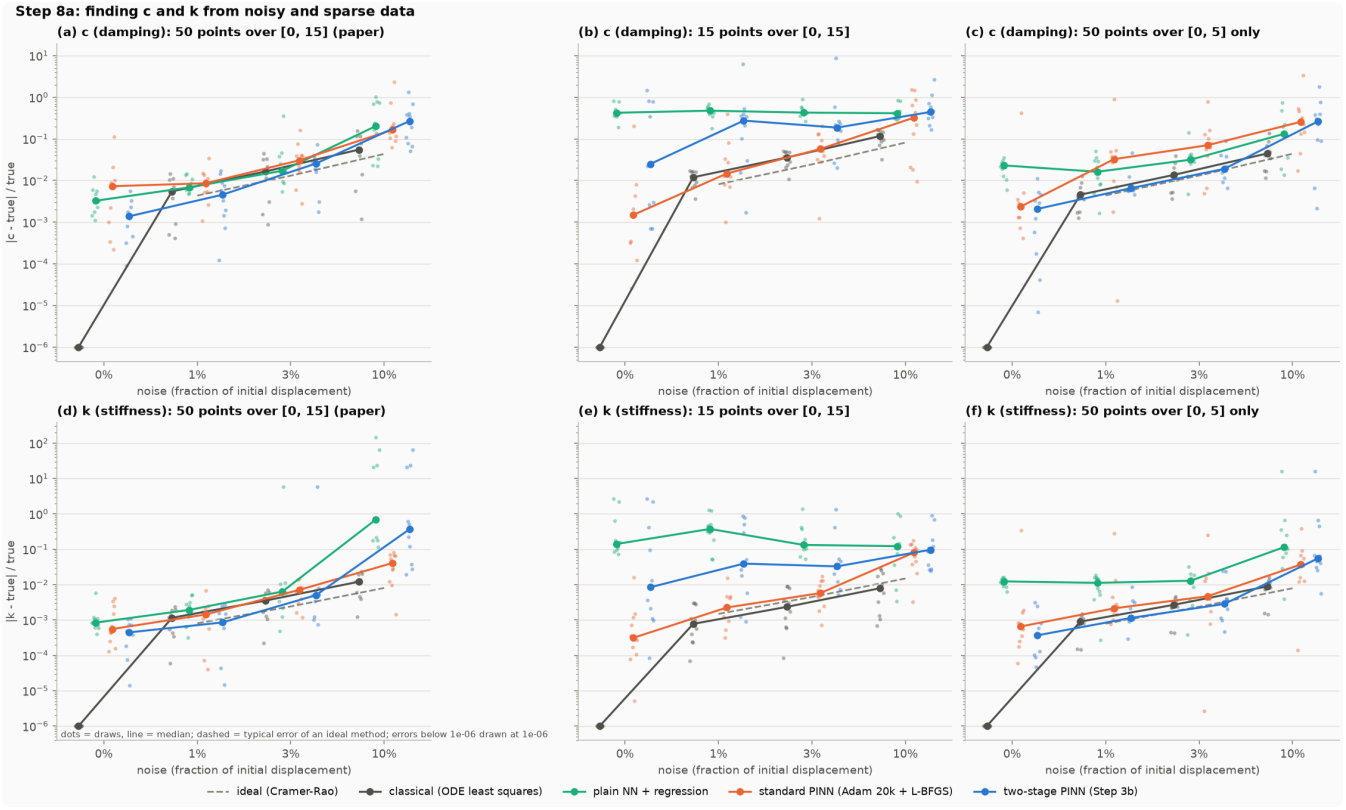


Figure 4. Noisy and sparse data. Error in c (top) and k (bottom) against noise for each data setting. Lines are medians over 10 draws, dots are individual draws, and the dashed line is the Cramer-Rao bound.

7. What we added to the paper

Area	The paper	Our work	Effect
Evidence	One run per experiment	5 to 10 seeds per configuration; median and worst case	Showed that 2 of 5 oscillator runs and most consolidation runs miss the paper's accuracy
Reference solutions	Fourier series (consolidation); finite elements, "no analytical solution" (heat)	Two independent exact solutions per forward problem; a series solution for heat, checked by our finite-difference solver	Errors measured exactly, down to 1e-9

Area	The paper	Our work	Effect
Error analysis	Error maps from one run	Error by region (corner, edge, early time); residual on fresh vs training points	Found where and why each PINN fails
Oscillator	Adam 100k; c found slowly and unreliably	Diagnosis via implied parameters; two-stage warm start	3/5 to 10/10 success; 30x faster
Consolidation	Output multiplied by x	Corner-enriched output $\mathrm{erf}(x / 2\sqrt{t}) + x$ t N; tested against adaptive sampling	About 200x lower error; no overshoot; 7x faster
Heat: edges	Soft penalty; exact boundary condition suggested but not tested	Distance-function output $(1 - x^2)(1 - y^2)$ N	30x lower error; no negative temperatures
Heat: corners	Not identified	Proof that every smooth network fails at the corners; exact corner terms	Corner hotspot removed; max error 3x lower again
Adaptive sampling	Future work	RAD implemented and tested	Little gain alone; small gain on top of enrichment
Robustness	Clean, dense data only	3 data settings x 4 noise levels x 10 draws; classical and plain-network baselines; Cramer-Rao bound	Showed when a PINN is worth using and when a classical fit is better
Engineering	Not described	float64; Adam to L-BFGS with divergence guard; per-term loss logs; 49 unit tests; resumable parallel runs	Bit-identical reruns on the same machine

8. Conclusions and limitations

A PINN's accuracy is decided by the few points where the physics is hardest: a corner, an edge, a slowly decaying tail. Averages and training loss hide these points. Building the known local behaviour into the output fixes them better than more iterations or smarter point placement.

Conclusions.

1. **One run is not evidence.** The paper's headline results hold for only some seeds.
2. **Training loss can mislead.** In consolidation and heat, the loss looked converged while the equation was badly violated near the corners. Checking the residual on fresh points exposed this every time.
3. **Put known structure in the output, not the loss.** Exact boundary conditions, the erf term and the corner terms each beat adaptive sampling and longer training. Each one also trained faster than the paper's setup, except the heat corner terms, which roughly double the time per run.
4. **Warm starts make inverse problems reliable.** Reading c and k off a data-only fit turned a 3-in-5 method into a 10-in-10 method, 30 times faster.
5. **A PINN is not always the right tool.** For a known ODE with two parameters, a classical least-squares fit is faster and more accurate. PINNs add value when data are incomplete and physics must fill the gaps.

Limitations.

- The corner and erf terms need analytical insight into each problem. They do not transfer automatically to new geometries.
- Uncertainty estimates (error bars for c and k) are planned but not yet done.
- All timings come from one laptop CPU (Intel i5-13450HX), so they compare methods rather than measure absolute speed.
- The study covers the paper's three problems only.

9. Reproducibility

Every number in this report comes from scripts in our repository, github.com/DBS13062006/pinn-mechanics.

- **Environment.** Python 3.12 and PyTorch, with CPU or CUDA. Setup instructions cover a Windows virtual environment, conda and Google Colab.
- **Tests.** 49 unit tests cover the engine, the exact solutions and the corner facts, and run in seconds.

- **Seeding.** Every run seeds all random generators. It also saves its library versions, hardware, loss history, weights and metrics next to the results.
- **Interrupted runs.** Long experiments run in parallel processes and can resume after an interruption without redoing finished runs.

Experiment	Runs	Parallel jobs	Wall time (laptop CPU)
Oscillator: reproduction	15	5	15 min
Oscillator: two-stage fix and ablations	40	5	5 min
Oscillator: noise and sparse data	360	4	50 min
Consolidation: reproduction	10	5	29 min
Consolidation: enrichment vs adaptive sampling	15	5	13 min
Heat: reproduction and exact boundary condition	15	3	82 min
Heat: corners	15	3	50 min

Hardware: Intel i5-13450HX (10 cores), 16 GB RAM, NVIDIA RTX 4050. These networks are small, so we ran them as parallel CPU jobs.

References

Base paper

1. Q. Zhang, Y. Chen, Z. Yang. *Data Driven Solutions and Discoveries in Mechanics Using Physics Informed Neural Network.* Stanford CS229 project report.

Physics-informed learning

2. M. Raissi, P. Perdikaris, G. E. Karniadakis. Physics-informed neural networks. *Journal of Computational Physics* 378, 686–707, 2019. The PINN framework and the Adam-then-L-BFGS training we follow.
3. I. E. Lagaris, A. Likas, D. I. Fotiadis. Artificial neural networks for solving ordinary and partial differential equations. *IEEE Transactions on Neural Networks* 9(5), 987–1000, 1998. Trial solutions that satisfy boundary conditions exactly; the basis of our hard-boundary output.

4. C. Wu, M. Zhu, Q. Tan, Y. Kartha, L. Lu. A comprehensive study of non-adaptive and residual-based adaptive sampling for physics-informed neural networks. *Computer Methods in Applied Mechanics and Engineering* 403, 115671, 2023. The RAD sampling we test.

Optimisation and networks

5. D. P. Kingma, J. Ba. Adam: a method for stochastic optimization. ICLR, 2015.
6. D. C. Liu, J. Nocedal. On the limited memory BFGS method for large scale optimization. *Mathematical Programming* 45, 503–528, 1989.
7. X. Glorot, Y. Bengio. Understanding the difficulty of training deep feedforward neural networks. AISTATS, 2010. Xavier initialisation.

Exact solutions and estimation theory

8. K. Terzaghi. *Theoretical Soil Mechanics.* Wiley, 1943. The consolidation equation.
9. H. S. Carslaw, J. C. Jaeger. *Conduction of Heat in Solids*, 2nd ed. Oxford, 1959. The erfc (method-of-images) solution used as our consolidation reference and as the erf enrichment term.
10. S. P. Timoshenko, J. N. Goodier. *Theory of Elasticity*, 3rd ed. McGraw-Hill, 1970. The series solution for torsion of a square bar, the same function as our heat reference.
11. J. M. Varah. A spline least squares method for numerical parameter estimation in differential equations. *SIAM Journal on Scientific and Statistical Computing* 3(1), 28–46, 1982. Fit-then-estimate (two-stage) parameter estimation, the idea behind our warm start.
12. S. M. Kay. *Fundamentals of Statistical Signal Processing: Estimation Theory.* Prentice Hall, 1993. The Cramer-Rao bound.